Takneek PS - Zenith

(100 Points)

The Big-GEM Theory

Introduction :

The development of modern Electronic Design Automation (EDA) tools is increasingly reliant on **GPU acceleration** to simulate massive **SystemVerilog** Register Transfer Level (RTL) netlists. Traditionally, software-based logic simulators running on CPUs face severe throughput bottlenecks when scaling to billions of gates. **NVIDIA's GEM architecture** introduced a breakthrough by reducing circuits to **1-bit boolean logic** and utilizing a **boomerang** scheduling layer to keep **SIMT** (Single Instruction, Multiple Thread) execution homogeneous, maximizing GPU utilization.

However, this approach forces **high-level hardware macros** to be shredded into thousands of **primitive logic gates** by the frontend synthesizer (**Yosys**). Shredding a single DSP multiplier destroys the architectural intent, inflates the graph depth by orders of magnitude, and results in catastrophic memory access overhead during simulation.

By directly modifying the open-source NVIDIA GEM codebase, specifically redesigning its **boomerang scheduling layer** and the underlying CUDA memory allocation to natively support a **heterogeneous execution graph**, these word-level macros can be simulated as native functional units. This prevents **warp divergence and memory stalls** while preserving the massive parallelism of the GPU, offering a highly optimized compiler framework for ultra-fast hardware simulation.

Takneek PS - Zenith

(100 Points)

Task/Problem Statement :

The challenge is to fork the official **NVIDIA GEM** simulator codebase (https://github.com/NVlabs/GEM) and architect a heterogeneous GPU-accelerated logic simulator. Teams must modify GEM's **existing** boomerang scheduling layer to natively evaluate word-level hardware macros without allowing the frontend to shred them into primitive bitwise gates.

At the software and theoretical level, the following must be modeled and implemented:

1. **Boomerang Scheduler Extension:** Modifying the Levelized Directed Acyclic Graph (DAG) scheduling equations to group and schedule mixed-width operations (1-bit boolean And-Inverter graphs executing alongside 48-bit arithmetic math) without stalling CUDA warps.
2. **Heterogeneous Memory Allocator:** Designing a memory packing strategy in GPU VRAM that maps 1-bit boolean states alongside 64-bit aligned contiguous memory blocks for arithmetic macros, enabling coalesced reads.
3. **Native Macro Evaluation**: Executing standard functional units on the GPU ALU using native “int64_t” types without SIMT warp divergence.

To ensure standardization across all participating halls, teams are strictly required to implement native GPU evaluation models for the following specific hardware primitives, mapped directly to standard Xilinx FPGA architectures:

A. The MAC Unit (DSP48E2 Simplified Subset):

To eliminate ambiguity regarding pipeline delays, the DSP model is constrained to the following precise configuration. All input/internal registers (AREG, BREG, CREG, DREG, ADREG, MREG) are strictly combinational (set to 0). **Only the final accumulator output register** (PREG) is clocked (set to 1). The logic must compute in a single pass using signed two's-complement arithmetic:

- **Pre-Adder Logic:** Implement the 27-bit D input, 27-bit A input, and 18-bit B input. The pre-adder computes AD = A + D or passes A directly.
- **Multiplier Logic:** Compute the 45-bit combinational product M = AD * B.
- **48-bit ALU & Control:** A 48-bit ALU that writes to the clocked P register. To avoid parsing the complex 9-bit Xilinx OPMODE, the Yosys parser must extract the intent and pass a simplified 2-bit state to the GPU kernel:
    - State 0: Bypass (P_next = C)
    - State 1: Multiply-Only (P_next = M)
    - State 2: Multiply-Accumulate (P_next = P_current + M)
- *Note on Output Pins*: You can safely **ignore** the dedicated OVERFLOW and UNDERFLOW output pins present on real DSP48E2 blocks. Only simulate the core 48-bit P output as defined above.

B. The Carry Chain (CARRY4 Primitive):

Teams must model the exact **Xilinx CARRY4** silicon block. The model must accept a 4-bit sum input S[3:0], a 4-bit data input DI[3:0], a cascade carry-in CIN, and a carry initialization bit CYINIT. It must compute the 4-bit carry-out CO[3:0] and the XOR'd result O[3:0] in a single GPU execution step using the following strict logic:

- C[0] = CYINIT | CIN (In valid RTL, only one of these is active).
- C[i+1] = (S[i] & C[i]) | (~S[i] & DI[i]) for i in 0 to 3.
- O[i] = S[i] ^ C[i] for i in 0 to 3.
- CO[i] = C[i+1] for i in 0 to 3.

C. Shift Register LUT (SRLC32E Primitive):

A 32-bit shift register that strictly follows clock-edge synchronization.

- **Inputs:** 1-bit Data (D), 1-bit Clock Enable (CE), 5-bit Address (A[4:0]).
- **Behavior:** On the global rising edge, if CE == 1, the internal 32-bit state shifts left (LSB to MSB), and D is loaded into index 0.
- **Outputs:** The read port natively outputs the bit at the dynamic address A[4:0] combinationally. The cascade port Q31 always outputs the bit at index 31 combinationally.

Takneek PS - Zenith

(100 Points)

Deliverables And Judging Criteria:

A. Host Parser & Yosys Pre-Processor (15 Points):

- **Modify** the GEM Yosys synthesis scripts (.ys files) to ingest **SystemVerilog** design and intercept the explicitly named **DSP48E2, CARRY4, and SRLC32E primitives**. Prevent the techmap and abc passes from flattening them into the And-Inverter Graph (AIG).
- **Extend GEM's host-side memory formatter** to map these intercepted topological nodes into flattened, 64-bit aligned CUDA memory buffers optimized for coalesced global memory bandwidth.

B. CUDA Execution Engine & Boomerang Modification (35 Points):

- Implement a fully integrated CUDA architecture that extends the boomerang kernel to evaluate macros directly on the GPU ALU.
- The topological scheduler must guarantee that macro-to-macro data dependencies (e.g., CO[3] of one CARRY4 feeding CIN of the next) are evaluated in the correct sequence without relying on intermediate boolean nodes.
- Utilize GPU Shared Memory and warp-level synchronization primitives (e.g., __shfl_sync()) to execute the macros with minimal thread divergence.

C. Hardware Macro Implementations (20 Points):

- **Cycle-accurate C++/CUDA models** for the DSP subset, CARRY4, and SRLC32E that respect the exact bit-level boundaries and register definitions outlined above. Teams must establish their own behavioral verification (via Vivado, Verilator, with SystemVerilog support enabled or custom Python testbenches) to prove structural accuracy. No golden reference code will be provided.

Takneek PS - Zenith

(100 Points)

D. Benchmarks & Performance Analysis (20 Points):

- **Total Simulation Throughput** measured in Simulated Cycles Per Second compared against unmodified GEM repository and the submission of other pools.
- **Profiling reports** detailing Memory Bandwidth Utilization and Warp Divergence metrics (via Nsight Compute).

E. Documentation & Reports (10 Points):

- **Mathematical definition** of the modified GEM scheduling equations used to manage the heterogeneous DAG.
- **Architectural block diagrams** mapping the FPGA primitives to the specific GPU memory hierarchy (Global vs. Shared vs. Registers).
- **Extensive numerical analysis** of the throughput gains.

Submission Format :

Submit your solution in zip format on the Google Form in the Discord Server later. The zip file should contain -

(1) Technical Report (PDF): Explanation of the modified scheduling equations, CUDA architecture, memory layout, and performance analysis. Mention benchmarking and numerical analysis in your report clearly and separately under a unique header.

(2) GitHub Link : Github URL to the team's Codebase.

(3) Testbenches & Golden Models: The CPU-based reference models and verification scripts the team authored to validate their native components.

(4) Benchmark Automation: Scripts used to generate performance logs via Nsight Compute.

(5) Source Code: The complete, properly structured source code of your simulator implementation.

Takneek PS - Zenith

(100 Points)

Submission Deadline: 3rd September 23:59.
Presentation Date: 4th September

Rules and Team Composition : Team will consist of at max **9** members
Maximum- 1 PHD/M.Tech, 1 Y23 B.Tech/BS, 1 Y24 B.Tech/BS, 3 Y25 B.Tech/BS
Minimum- 3 Y26 B.Tech/BS

Note :

(1) Partial submissions (e.g., implementing only the DSP and CARRY4 correctly) are **allowed** and will be graded proportionally based on throughput gains achieved.
(2) Clocking Constraint: Assume a **single** global clock domain for the entire netlist. All synchronous macros (PREG in DSP, shifting in SRLC32E) trigger on the exact same rising edge.
(3) Initialization Constraint: Assume all internal macro registers **initialize to zero.** Parsing INIT hex strings from the netlist is **NOT** required
(4) Yosys Version & SystemVerilog Support: All baseline and modified benchmarks must be run using the newest stable version of Yosys, **i.e. Yosys 0.68**. Your synthesis scripts must successfully parse **IEEE 1800-2012 SystemVerilog syntax.** You are permitted to rely on **standard Yosys SystemVerilog integrations** (like the Slang-based frontend) to process the hidden benchmarks. Ensure your parser scripts are compatible with this version's JSON netlist output.
(5) CUDA Libraries: You may use standard libraries included in the CUDA Toolkit (like Thrust for sorting/memory operations), but the core Boomerang scheduling loop and macro evaluation must be your own custom CUDA kernel to demonstrate architectural understanding.
(6) Final performance and throughput evaluation will be conducted on hidden benchmark **SystemVerilog** netlists, and a standardised testing machine, provided by the judging panel.

(For Any Queries, The Pool Captains and PS Leads are encouraged to ask in the Discord channel.)